Case Study 05: Water Tank Inflow-Outflow & Level Monitoring System

1. Problem Statement

Develop a Python program to track and monitor the water volume inside a storage reservoir over a 3-day observation window. The system starts with an initial baseline quantity and observes daily inflow and outflow volumes. It must enforce upper bounds (maximum storage capacity) and lower bounds (minimum capacity threshold of zero litres), trigger automated warnings when overflow or shortage situations occur, compute end-of-day balances, and pinpoint the exact day on which a designated capacity target is first reached.

2. Objectives

- To simulate state-based fluid balance dynamics using fixed-iteration for loops across multiple discrete intervals.
- To implement conditional clamping to maintain reservoir water levels strictly within physical limits ($[0,\text{capacity}]$).
- To detect and announce system anomalies such as tank overflow and critical shortage states.

- To record the earliest day a threshold target is reached using sentinel flag/latching logic (None state tracking).

3. Assumptions

- The system begins with an initial water level of 40 litres, a maximum tank capacity of 100 litres, and a tracking target threshold of 80 litres.

- Daily inflow and outflow measurements are entered as integer quantities.

- If net water level exceeds capacity (100 litres), excess fluid is lost to overflow and the level is capped at 100 litres.

- If net water level falls below zero, fluid cannot be negative; a shortage is reported and the balance is clamped to 0 litres.

- The target achievement day is locked when first met during the 3-day cycle and is not overwritten by subsequent fluctuations.

4. Algorithm

- **Step 1:** Start the program.

- **Step 2:** Initialize simulation constants and state variables: level = 40, capacity = 100, target = 80, and target_day = None.

- **Step 3:** For each day in range(1, 4) (Day 1 through Day 3):

    - Prompt and read integer inputs for inflow and outflow.

    - Update current reservoir volume: level = level + inflow - outflow.

    - If level > capacity: print "Overflow detected" and clamp level = capacity.

    - If level < 0: print "Shortage detected" and clamp level = 0.

    - Display the updated end-of-day level: "Water level: <level> litres".

    - If level >= target and target_day is None: assign target_day = day.

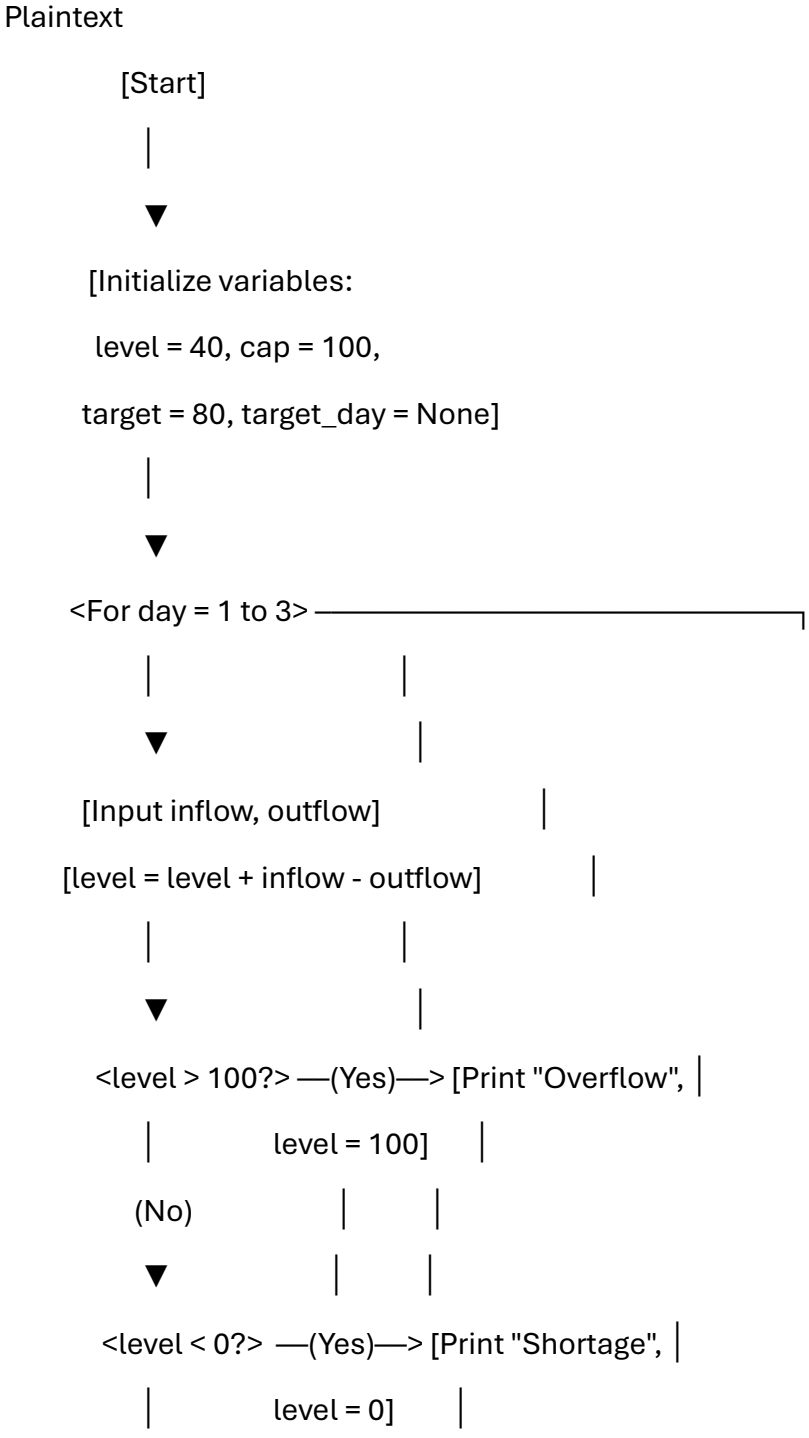
- **Step 4:** After completing the 3-day loop:

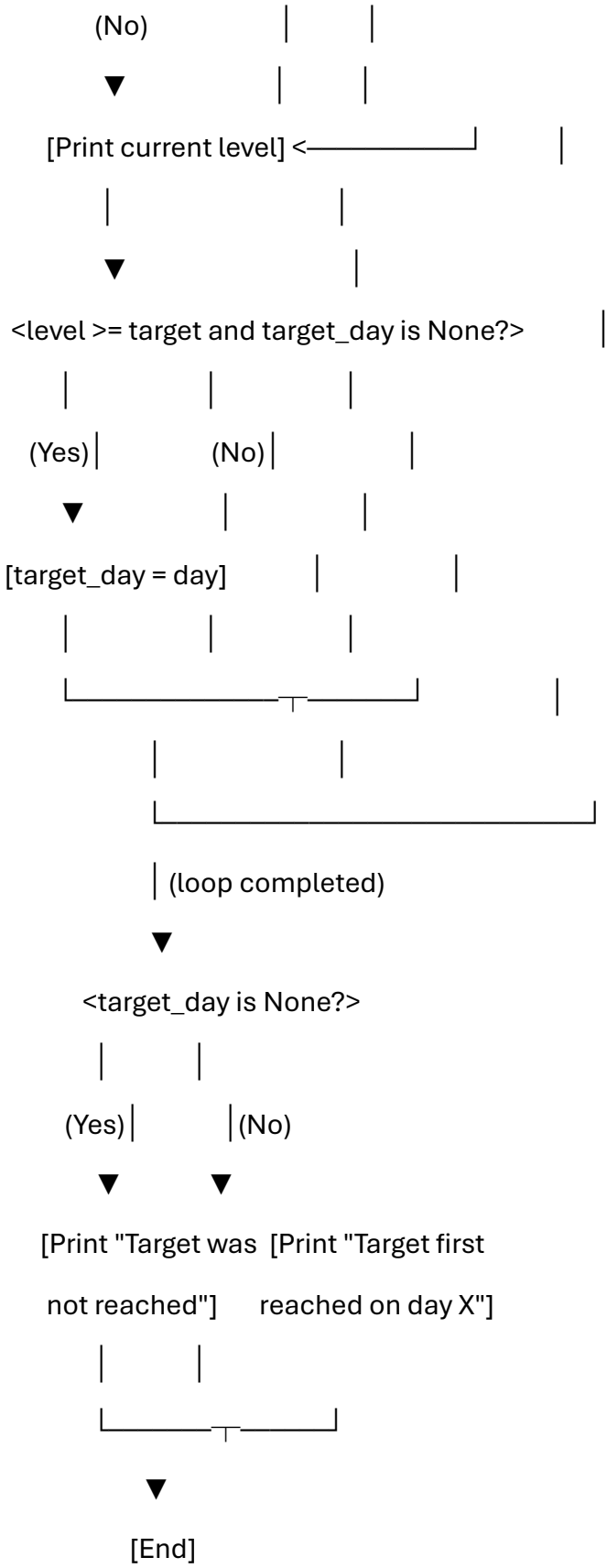
    - If target_day is None, print "Target was not reached".

    - Otherwise, print "Target first reached on day <target_day>".

- **Step 5:** Stop the program.

5. Flowchart Logic / Diagram




(No)
[Print current level]
<level >= target and target_day is None?>
(Yes)
(No)
[target_day = day]
(loop completed)
<target_day is None?>
(Yes)
(No)
[Print "Target was not reached"]
[Print "Target first reached on day X"]
[End]

6. Pseudocode

```
Plaintext
BEGIN
  SET level = 40
  SET capacity = 100
  SET target = 80
  SET target_day = NULL

  FOR day FROM 1 TO 3 DO
    PRINT "Day " + day + " inflow (litres): "
    READ inflow
    PRINT "Day " + day + " outflow (litres): "
    READ outflow

    level = level + inflow - outflow

    IF level > capacity THEN
      PRINT "Overflow detected"
      level = capacity
    END IF

    IF level < 0 THEN
      PRINT "Shortage detected"
      level = 0
```

```
    END IF

    PRINT "Water level:", level, "litres"

    IF level >= target AND target_day IS NULL THEN
      target_day = day
    END IF
  END FOR

  IF target_day IS NULL THEN
    PRINT "Target was not reached"
  ELSE
    PRINT "Target first reached on day", target_day
  END IF
END
```

7. Python Source Code

Python

```
# Water Tank Inflow-Outflow & Level Monitoring System

# System baseline specifications
level = 40
capacity = 100
target = 80
```

```
target_day = None

# Simulate tank operations over 3 days
for day in range(1, 4):
    inflow = int(input("Day " + str(day) + " inflow (litres): "))
    outflow = int(input("Day " + str(day) + " outflow (litres): "))
    level = level + inflow - outflow

    # Boundary enforcement for overflow
    if level > capacity:
        print("Overflow detected")
        level = capacity

    # Boundary enforcement for shortage
    if level < 0:
        print("Shortage detected")
        level = 0

    print("Water level:", level, "litres")

    # Latch target achievement day
    if level >= target and target_day is None:
        target_day = day

# Final status evaluation
if target_day is None:
    print("Target was not reached")
else:
```

```
print("Target first reached on day", target_day)
```

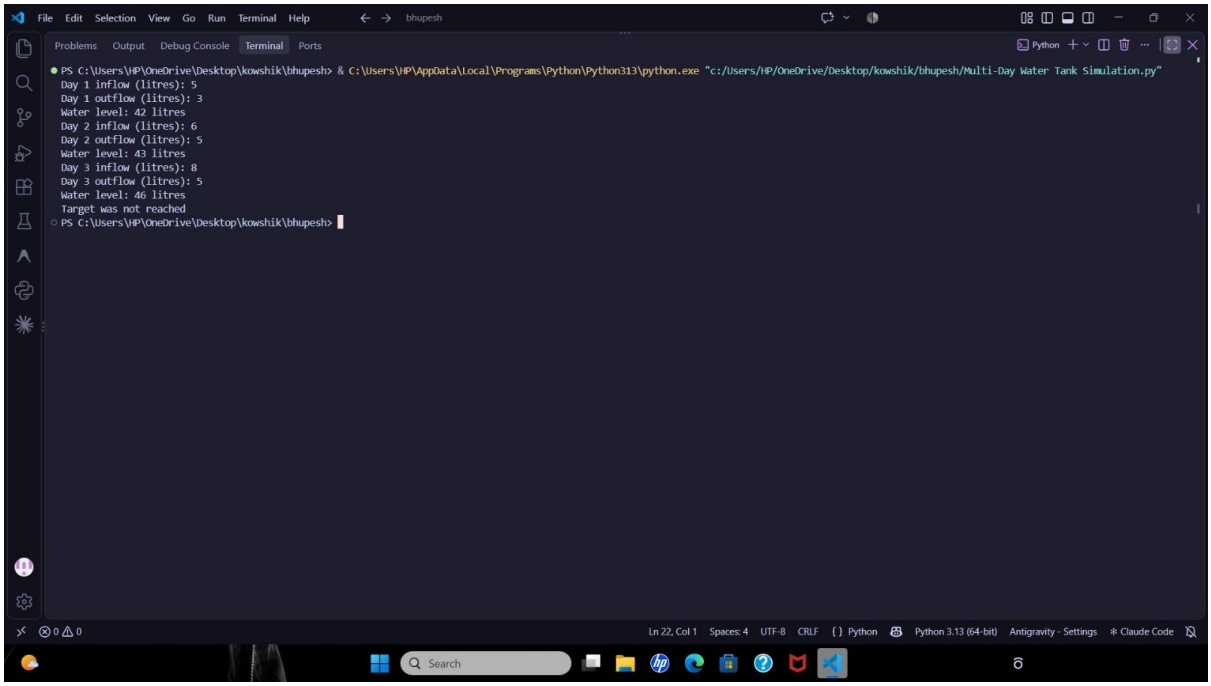
8. Test Cases

Test Case ID	Input Sequence (Inflow, Outflow)	Expected Output	Actual Output	Status
TC_01 (Target Reached Day 2)	Day 1: 30, 10 Day 2: 30, 5 Day 3: 10, 20	Day 1: Level 60 Day 2: Level 85 Day 3: Level 75 Target first reached on day 2	Water level: 60 litres Water level: 85 litres Water level: 75 litres Target first reached on day 2	Pass
TC_02 (Overflow on Day 1)	Day 1: 80, 10 Day 2: 10, 20	Day 1: Overflow detected, Level 100	Overflow detected	Pass

Test Case ID	Input Sequence (Inflow, Outflow)	Expected Output	Actual Output	Status
	Day 3: 5, 15	Day 2: Level 90 Day 3: Level 80 Target first reached on day 1	Water level: 100 litres Water level: 90 litres Water level: 80 litres Target first reached on day 1	
TC_03 (Shortage on Day 1 & Target Not Reached)	Day 1: 10, 60 Day 2: 20, 10 Day 3: 15, 5	Day 1: Shortage detected, Level 0 Day 2: Level 10 Day 3: Level 20	Shortage detected Water level: 0 litres Water level: 10 litres	Pass

Test Case ID	Input Sequence (Inflow, Outflow)	Expected Output	Actual Output	Status
		Target was not reached	Water level: 20 litres Target was not reached	
TC_04 (Flips Above Target Multiple Days)	Day 1: 50, 10 Day 2: 10, 5 Day 3: 15, 10	Day 1: Level 80 Day 2: Level 85 Day 3: Level 90 Target first reached on day 1	Water level: 80 litres Water level: 85 litres Water level: 90 litres Target first reached on day 1	Pass

9. Output Screenshots



10. Conclusion

The Water Tank Inflow-Outflow & Level Monitoring System was executed and verified successfully. The script demonstrates iterative boundary clamping, conditional state updates, threshold latching logic using Python's None sentinel type, and multi-day status tracking.